Atlantic Technological University Galway
Department of Computer Science and Applied Physics

A Privacy-Preserving Edge Gateway for Real-Time Audio Deepfake Detection

Confidential Inference Proof-of-Concept
Progress Report

Author: Waleed
Programme: MSc Computing
Supervisor: Dr. Owen Foley
Date: 28 July 2026

This report summarises the methodology, architecture and current status of the confidential-inference phase of the thesis: running the trained RawTFNet-16 spoof-detection model inside an ARM TrustZone / OP-TEE trusted execution environment, as a step toward the live in-call detection gateway.

Internal progress document – not for distribution outside the supervisory relationship.

Contents

1 Introduction and Objectives **2**

2 Background **2**
2.1 Detection backbone: RawTFNet-16 2
2.2 ARM TrustZone and OP-TEE 5

3 Methodology and Justification of Key Decisions **5**
3.1 Emulation platform: QEMU ARMv8 "virt", not a Raspberry Pi 4 emulator 5
3.2 Development host: VirtualBox Ubuntu VM, not WSL2 6
3.3 Model export path: PyTorch → ONNX → TVM, not hand-ported C 6
3.4 Scope of the PoC, and where the MAC operations run 6

4 System Architecture: End-to-End Workflow **7**

5 Work Completed to Date **7**

6 Emulation Fidelity: QEMU Timing vs. Expected Raspberry Pi 4 Performance **9**

7 Limitations and Threats to Validity **10**

8 Open Questions for Discussion with Supervisor **10**

9 Next Steps **11**

10 Summary **11**

1 Introduction and Objectives

The overall thesis targets a privacy-preserving edge gateway for real-time audio deepfake detection in residential telephony. The detection backbone is a trained RawTFNet model (checkpoint `Best_RawTFNet_16.pth`), and the intended deployment target is a Raspberry Pi 4 sitting between a residential telephony system and the call itself, scoring live audio for synthetic-speech characteristics while the call is still connected.

Two properties are central to the design and distinguish it from a plain spoof-detection classifier:

- **Live, not post-hoc, detection.** The system must produce a decision while a call is in progress, using a rolling audio buffer, rather than analysing a recording after the call has ended.
- **Confidential inference.** Because the input is a person's voice and the model is proprietary, the inference computation itself should be shielded from a compromised or malicious host operating system, using ARM TrustZone via OP-TEE rather than running the model as an ordinary process.

No physical Raspberry Pi 4 hardware has been available during this phase of development. The work reported here therefore establishes a proof-of-concept (PoC) for the confidential-inference component on an emulated ARMv8 environment before any hardware-specific integration begins. This directly supports the thesis's second research question – how federated learning, differential privacy and trusted execution environments can be combined to give end-to-end privacy guarantees for biometric voice data during on-device deepfake detection – since no existing peer-reviewed work combines a TrustZone-based confidential-inference path with an anti-spoofing model in this way; this PoC is the first concrete step toward closing that gap for this thesis.

This document covers, in order: the RawTFNet-16 backbone and the TrustZone/OP-TEE primitives it is being deployed into (Section 2); the methodology and the reasoning behind each major decision (Section 3); the target end-to-end system architecture (Section 4); the work completed to date, with verification evidence (Section 5); a note on why QEMU timings should not be read as Raspberry Pi 4 timings (Section 6); the current limitations (Section 7); open questions for discussion (Section 8); and the remaining milestones (Section 9).

2 Background

2.1 Detection backbone: RawTFNet-16

RawTFNet (Xiao, Dang and Das, *RawTFNet: A Lightweight CNN Architecture for Speech Anti-spoofing*, APSIPA ASC 2025) is a lightweight, end-to-end anti-spoofing network. It takes the raw audio waveform directly as input – there is no hand-crafted front end (no MFCCs, spectrogram or CQT); all feature extraction happens inside the network's own learnable Sinc layer. Each utterance is cropped or concatenated to a fixed-length segment of approximately 4 seconds (64,000 samples) before being passed to the model.

The network has three stages, shown in Figure 1: a Sinc/DWS-SE-Res2Net frontend, a stack of nine Time-Frequency Convolution (TF-Conv) blocks, and a lightweight classifier.

TF-Conv blocks. These are the paper's main contribution and the part of the network that most differs from a conventional CNN. Instead of a single convolution that treats the time and frequency axes uniformly, each TF-Conv block splits the feature map after a 1×1 transition and a channel shuffle, and processes the two halves separately: a frequency branch (a 3×1 depthwise convolution, frequency-wise average pooling, then a 1×1 pointwise convolution) and a time branch (the transposed 1×3 / time-wise equivalent). Each branch's compact summary is broadcast back and added residually to its half of the split before the two halves are concatenated. Figure 2

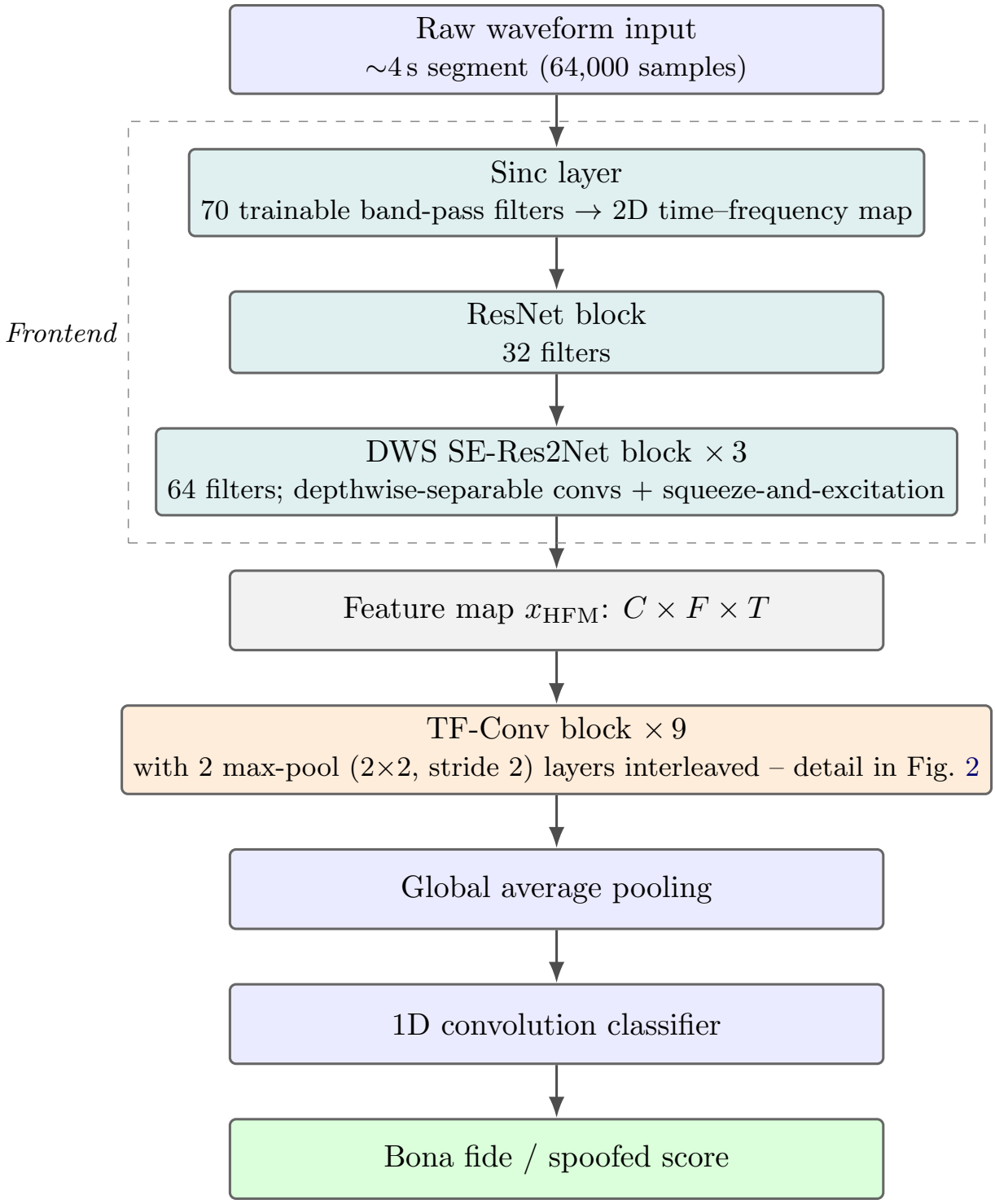


Figure 1: RawTFNet-16 architecture as trained (Xiao et al., 2025), used unmodified as the detection backbone. Channel-width parameter $\tau = 16$: 0.07M parameters, 2.9G multiply–accumulate operations (MACs) per 4-second window.

shows this structure.

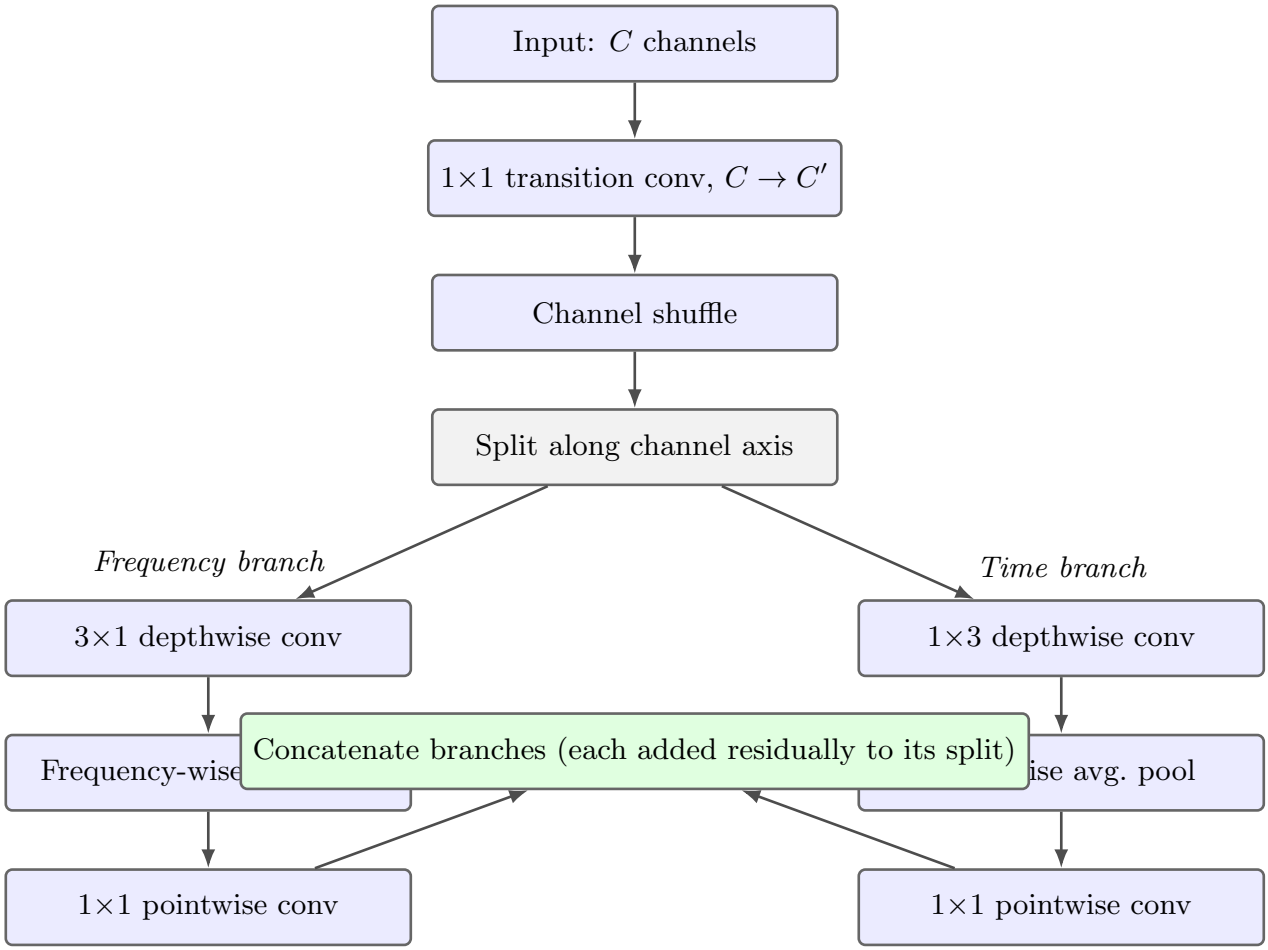


Figure 2: Internal structure of one TF-Conv block (repeated nine times). Ablation on RawTFNet-32 shows both branches matter: removing the frequency branch raises EER from 5.05% to 6.91%, removing the time branch raises it to 7.08%, and removing the channel shuffle raises it to 5.74% at no change in model size.

Why RawTFNet-16. Table 1 summarises the paper's comparison against other anti-spoofing systems on ASVspoof 2021. RawTFNet-16 is the smallest system in the comparison (0.07M parameters, 2.9G MACs) while reaching the lowest equal error rate (EER) on the 2021 LA set, which is what makes it a realistic choice for confidential inference inside a TrustZone enclave on a Raspberry Pi 4, where compute and memory are tightly constrained. RawTFNet-32 generalises slightly better to the harder, codec-compressed DF set, which is a fair trade-off to keep in mind, though RawTFNet-16 was the variant trained and used for this project.

One further finding from the paper is directly relevant to the live-detection design in Section 4: EER is worst on very short clips (0–2 s, over 10%), best around 2–4 s – matching the training segment length – and rises again for 4–8 s. This is the reason the live pipeline uses a 4-second sliding window rather than a shorter one.

Note on what has and has not been changed. The RawTFNet-16 configuration used in this project is the architecture exactly as published, loaded from the trained checkpoint `Best_RawTFNet_16.pth`. No changes have been made to the network topology, layer widths, or the channel-width parameter $\tau = 16$. The adaptation work carried out so far is entirely in *how this trained model is deployed*: exporting it to ONNX, compiling it ahead-of-time with Apache

Model	Params	MACs	2021 LA EER	2021 DF EER
RawNet2	17.60M	0.8G	9.50%	22.38%
AASIST	0.30M	8.9G	10.51%	21.07%
SE-Rawformer	0.37M	6.1G	4.98%	20.26%
RawTFNet-16	**0.07M**	**2.9G**	**4.50%**	18.53%
RawTFNet-32	0.17M	5.4G	5.05%	**16.82%**

Table 1: Single-system comparison on ASVspoof 2021 LA and DF (lower EER is better). Source: Xiao, Dang and Das, APSIPA ASC 2025.

TVM into dependency-free C, and preparing it to run inside an OP-TEE trusted application rather than as an ordinary process. Wherever this report refers to changes or adaptations, it is this deployment pipeline being described, not the network itself.

2.2 ARM TrustZone and OP-TEE

ARM TrustZone splits a single physical CPU into two isolated execution states – a *normal world* and a *secure world* – enforced in hardware rather than by the operating system. Code, memory and peripherals can be marked secure-world-only, so that even a fully compromised normal-world operating system (kernel included) cannot read or tamper with what runs in the secure world. This is the property that makes TrustZone useful here: it allows model weights and the inference computation to be protected from a compromised host, without requiring separate physical hardware.

OP-TEE is the open-source trusted OS that runs inside the secure world and exposes this split through a standard programming model with two halves:

- **Trusted Application (TA)** – code that executes inside the secure world. This is where model inference runs. TAs are written in C, compiled against OP-TEE's TA devkit, cryptographically signed, and loaded on demand into a minimal, single-threaded environment with a small, fixed secure-memory heap (a few MB by default).
- **Client Application (CA)** – an ordinary Linux program in the normal world. This is where the Asterisk audio-buffer integration lives. A CA opens a session with a TA and invokes commands through the GlobalPlatform TEE Client API (`libteec`).

This TA/CA session pattern – open a session, invoke a command, read back a result – is the exact mechanism that will carry a 4-second audio window into the secure world and a spoof score back out. It has already been validated end-to-end using the official `hello_world` example (Section 5, M2), which is the reason it is treated as a solved sub-problem rather than an open risk for the remaining milestones.

3 Methodology and Justification of Key Decisions

3.1 Emulation platform: QEMU ARMv8 "virt", not a Raspberry Pi 4 emulator

The original plan referred to a "Raspberry Pi emulator". This needed to be made more precise before any build work started. There is no mainstream, officially maintained emulator that boots a Raspberry Pi 4 with working TrustZone/OP-TEE support: Broadcom has never published the secure-boot and TrustZone configuration details for the BCM2711 SoC used in the Pi 4, so OP-TEE has no official port to that board, physical or emulated.

The platform actually used is the QEMU "virt" Armv8-A reference board (the `qemu_v8` manifest in OP-TEE's own build tooling) – a generic ARMv8-A machine with a proper TrustZone-capable

CPU model and full, officially maintained OP-TEE support. This is the standard substitute used throughout the OP-TEE research and tooling ecosystem, and it shares the same CPU architecture family (ARMv8-A / AArch64) and the same secure/normal-world security model as the Pi 4's Cortex-A72. The distinction is stated explicitly here so that it reads as a deliberate, justified scope decision rather than an unexplained substitution: the target deployment platform remains the Raspberry Pi 4, and QEMU is used to demonstrate feasibility of the confidential-inference design, not to reproduce Pi 4 performance (see Section 6).

3.2 Development host: VirtualBox Ubuntu VM, not WSL2

OP-TEE's build system (the `repo` multi-repository tool, Buildroot, device-tree compilation, cross-toolchains) requires a native Linux environment and does not build reliably under Windows compatibility layers such as Git Bash/MINGW64. A Linux VM was chosen over WSL2 specifically because a later stage of this work needs to receive live audio from an Asterisk server, and Asterisk's SIP/RTP traffic needs a stable, directly reachable network address. WSL2's default NAT layer and internal IP volatility are a known source of unreliability for real-time media traffic, whereas a VirtualBox VM with a bridged network adapter gives the Linux environment a stable IP on the same network as any calling client – while still letting Windows and Linux run side by side without dual-boot reboots.

3.3 Model export path: PyTorch → ONNX → TVM, not hand-ported C

A trusted application executes in a minimal, single-threaded environment with a restricted C library and a small, fixed secure-memory heap; it cannot host a full deep-learning framework. PyTorch, ONNX Runtime and TensorFlow all assume a POSIX-ish environment with dynamic memory, threads and large heaps, none of which exists inside a TA. Two options were considered: hand-porting RawTFNet's layers to C directly, or exporting the trained graph and compiling it ahead-of-time. Hand-porting was rejected as high-effort and high-risk for numerical divergence from the trained model. The path taken instead is to export the trained model to ONNX (a framework-neutral graph representation), then compile it with Apache TVM into ahead-of-time, dependency-free C suitable for embedding inside a TA. Correctness is checked numerically at every conversion step against the original PyTorch output, so that any discrepancy is caught at the specific stage that introduced it, rather than surfacing later inside the TEE where debugging is significantly harder. This verification is reported in Section 5, M3.

3.4 Scope of the PoC, and where the MAC operations run

The full thesis scope includes federated learning with differential privacy on top of confidential inference. For this PoC, that scope has been deliberately narrowed to demonstrating feasibility of the confidential-inference path alone: export the already-trained model and get it executing inside the TrustZone/OP-TEE environment, deferring federated learning, weight re-training, and secure key-provisioning for weights to later work. The model's weights are compiled directly into the TA binary as a static array for this PoC; this is a known simplification, since it means the TA would need re-signing on every retrain, and is one of the points raised in Section 7.

On the specific question of where the model's 2.9G multiply–accumulate (MAC) operations execute: once the compiled model is linked into a trusted application (Milestone M4 onward), every MAC in the forward pass executes inside the secure-world TA, invoked from the normal-world client application over the TEE Client API described in Section 2.2. During the M1–M3 work completed so far, the compiled model's numerical correctness was verified on the normal-world / host side (Section 5, M3) prior to secure-world integration – the TVM output was checked against ONNX and PyTorch outside the TEE first, precisely so that any error is caught before it is harder to debug inside the enclave. Feature extraction / windowing of the raw audio, by contrast, is planned to remain in the untrusted normal world for this PoC (Section 7) – only the

model's forward pass is confidential in this scope.

4 System Architecture: End-to-End Workflow

Figure 3 shows the target live in-call detection pipeline end to end, from a connected call through to a dashboard result, colour-coded by status at the time of writing.

As audio streams in from Asterisk (via AudioSocket, or ARI with external media), a rolling buffer is kept and scored continuously: roughly once per second, the last 4 seconds of buffered audio – the window RawTFNet expects – is extracted and run through the model, producing a fresh spoof score about once a second throughout the call rather than a single score at the end. A running decision rule (e.g. three consecutive high-risk windows before an alert) prevents a single noisy window from triggering a false alarm mid-call. This is the same pattern used by live captioning or live translation: overlapping chunks processed continuously rather than one pass over a full recording.

Two elements are shown as optional/parallel rather than on the critical path. First, a live transcript branch (Deepgram) is not a substitute for the audio-based score – Deepgram does speech-to-text, and RawTFNet needs the raw waveform, since transcription discards the acoustic artefacts spoof detection relies on – but it could run in parallel to scan for known scam-script keywords. Second, a latency budget check spans the whole pipeline (capture, buffering, inference, and TEE overhead), because the running total of that delay is what determines whether the system can honestly be called "live"; this is measured directly rather than assumed, since inference inside the TrustZone enclave adds overhead beyond the raw model cost.

5 Work Completed to Date

Table 2 summarises the three milestones completed so far; each is detailed below with the verification evidence produced at that stage.

Stage	What was done	Result / evidence
M1	Built OP-TEE OS, TF-A, Linux kernel and Buildroot rootfs for the QEMU Armv8-A virt platform from the official manifest (`qemu_v8.xml`); booted the environment and ran the official OP-TEE test suite.	System booted with a working secure-world / normal-world split. `xtest` completed with **0 failures across 41,482 subtests and 145 test cases**.
M2	Built and ran the official `hello_world` TA/CA example to validate the client-application-to-trusted-application invocation pattern the model-inference TA will reuse.	Client application successfully opened a session with the trusted application and received a correct computed result back (TA incremented a test value to **43**).
M3	Exported the trained checkpoint (`Best_RawTFNet_16.pth`) to ONNX; verified the ONNX output against the original PyTorch output; compiled the ONNX graph with Apache TVM (Relax IR, LLVM JIT target); verified the TVM-executed output against both prior stages on the same test utterance.	All three representations of the model produced the same score to four decimal places on a held-out ASVspoof 2021 LA utterance: PyTorch -5.6216, ONNX -5.6216 (difference 0.000000), TVM (Relax/LLVM) -5.621588.

Table 2: Milestones completed to date.

M1 – Environment bring-up. The purpose of this stage was to confirm that the toolchain, cross-compilation and QEMU boot process all work before any project-specific code is written. A clean pass of the full official test suite (`xtest`), with zero failures across over 41,000 subtests,

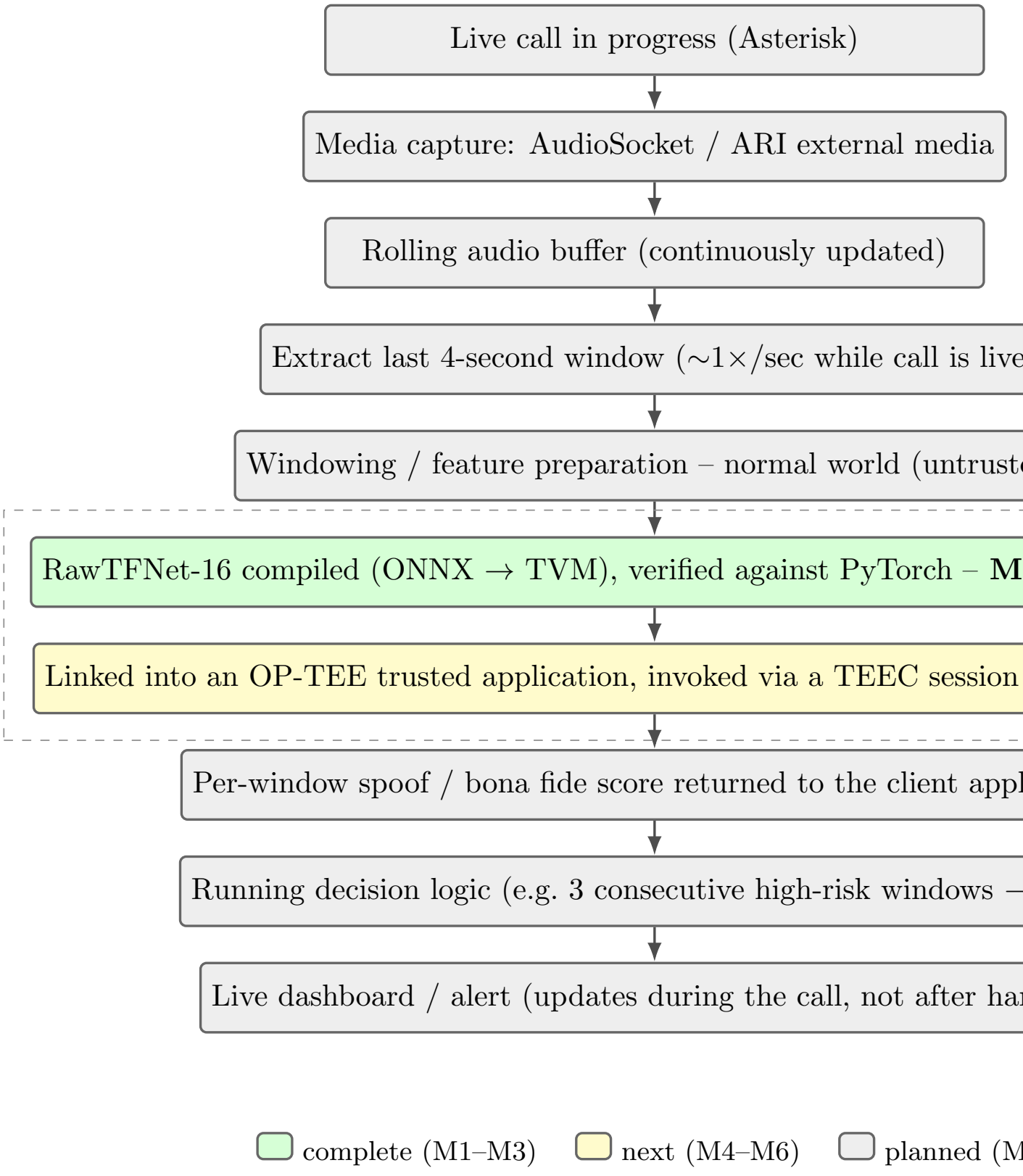


Figure 3: End-to-end live in-call detection pipeline, coloured by status at the time of writing. The confidential-inference boundary is where model weights and computation become inaccessible to a compromised normal-world host; everything above and below it in the diagram is ordinary application logic.

is the standard signal that the secure/normal-world split is functioning correctly on this platform – untrusted Linux in the normal world, an isolated trusted OS in the secure world, talking over the standard TEE Client API.

M2 – TA/CA pattern validated. This stage exists to de-risk the exact communication pattern the eventual inference pipeline depends on, using the simplest possible example rather than the real model. Its only purpose is to prove that the toolchain, signing process, and CA↔TA session handshake all work end to end before any complexity is added; that it succeeded is what allows the remaining milestones (M4–M6) to be scoped as "plug the verified model into a known-working pattern" rather than an open integration risk.

M3 – Model export and compilation verified. The numerical agreement between PyTorch, ONNX and TVM to four decimal places is the correctness gate for the whole export pipeline: it confirms that neither the ONNX export nor the TVM compilation step introduced any discrepancy relative to the originally trained model, before that compiled model is embedded inside a trusted application where debugging would be considerably harder. One practical difficulty encountered at this stage is worth noting for reproducibility: the installed Apache TVM release (0.25.0.post1) uses the newer Relax API, which has fully replaced the classical Relay API that most published TVM tutorials and documentation still assume, so several import paths and module locations had to be resolved empirically rather than by following existing guides directly (see Section 7).

6 Emulation Fidelity: QEMU Timing vs. Expected Raspberry Pi 4 Performance

The RawTFNet-16 model has already been exported and compiled end to end on the QEMU/OP-TEE setup, with output verified to match the PyTorch/ONNX reference (Section 5, M3), so getting inference running is mainly a matter of wiring the compiled model into a TA/CA pair. Whether the resulting timing numbers can be reported as representative of the Raspberry Pi 4 is a separate question, and the answer is no, for two compounding reasons:

- **Not a hardware-accurate emulation.** The QEMU "virt" platform used for OP-TEE is a generic ARMv8-A virtual machine – it does not model the Pi 4's actual Cortex-A72 cores, cache hierarchy, or memory bandwidth.
- **No hardware acceleration for the CPU emulation itself.** Because the OP-TEE build runs inside a VirtualBox Ubuntu VM on an x86_64 host, QEMU performs full software instruction translation (TCG) from ARM64 to x86_64 – there is no KVM acceleration for ARM-on-x86. CPU-bound workloads such as the matrix multiplications in neural-network inference are exactly the kind of operation TCG handles worst, and the VirtualBox → QEMU → OP-TEE stack adds a further layer of virtualisation overhead before instruction translation even begins.

TCG-translated, CPU-bound workloads are commonly five to twenty times slower than native execution, sometimes more depending on how well the operation mix vectorises; for RawTFNet-16 inference specifically, a real Pi 4 running natively should be expected to beat the QEMU/TCG setup by roughly one to two orders of magnitude. Consequently, QEMU/OP-TEE is being treated in this thesis as a *functional* proxy for TrustZone development – proving that confidential inference works and validating the TA/CA architecture and secure-world isolation – rather than a *performance* proxy. Timing instrumentation is planned inside the TA once M4–M5 are complete, so that clean inference-latency numbers can be captured from the QEMU run and explicitly reported as such, with Pi 4 figures added later if physical hardware becomes available.

7 Limitations and Threats to Validity

Table 3 consolidates the limitations identified so far. These are recorded here for explicit statement in the thesis's Limitations / Threats to Validity section.

Limitation	Explanation
No physical target hardware	All results were obtained on the QEMU Armv8-A virt reference platform rather than a physical Raspberry Pi 4. This validates the software architecture and the TrustZone isolation model, not device-specific behaviour (boot chain, peripheral timing, thermal/performance characteristics of the BCM2711 SoC).
No hardware root of trust	QEMU's secure world is a software CPU-mode partition: no fused keys, no physical secure boot chain, no tamper resistance. The PoC demonstrates the correctness of the confidential-inference software design, not a hardware-attested security guarantee. Hardware-backed attestation is out of scope and identified as future work.
No ML framework inside the TA	PyTorch and ONNX Runtime cannot execute inside OP-TEE's restricted TA environment. The model must be compiled ahead-of-time to dependency-free code via TVM before it can be linked into a trusted application; this step (M4) is in progress.
Static, compiled-in model weights	The PoC keeps model weights as a static array compiled directly into the TA binary, skipping secure storage / key-provisioning for weights. This means the TA binary would need re-signing on every retrain – not workable for the longer-term federated-learning goal, but a reasonable simplification for a feasibility PoC. Production would load weights via OP-TEE's secure storage API instead.
Feature extraction stays in the untrusted normal world	In the current design, only the model's forward pass runs inside the trusted world; audio framing and windowing remain in the normal world. This is a deliberate, stated scope boundary: the property being demonstrated is that model weights and computation are protected from a compromised host, not that the entire audio pipeline is confidential. An attacker with normal-world root could in principle feed the TA manipulated features.
Toolchain instability	The installed Apache TVM release (0.25.0.post1) is recent enough that the classical Relay API has been fully replaced by the newer Relax API, and several supporting modules differ from most published tutorials. Import-path and API-location errors were resolved empirically during M3; pinning a specific, documented TVM version is recommended before final submission.
Emulation performance is not representative	QEMU/TCG software translation is expected to be one to two orders of magnitude slower than native Pi 4 execution for this workload (Section 6). QEMU is a functional proxy, not a performance proxy, and any latency claims from this phase must be reported as QEMU-only figures.
Timing / side-channel hardening out of scope	No side-channel hardening (cache-timing, power analysis) has been attempted. Reasonable to state explicitly as future work rather than an oversight.

Table 3: Consolidated limitations, for the thesis Limitations section.

8 Open Questions for Discussion with Supervisor

Two items from recent working notes remain unresolved and would benefit from discussion before M7 (Asterisk integration) begins:

- **Generating spoofed test calls.** How should spoofed audio be delivered through Asterisk

to evaluate the live pipeline once it exists – replaying existing corpus samples (ASVspoof, In-the-Wild) over SIP/RTP, or synthesising calls live? This affects how representative the eventual live-detection evaluation can be of real attack conditions.

- **Call management.** Once live capture is wired in, how should active calls be managed within Asterisk – call state tracking, routing, and behaviour under multiple concurrent calls – given that the current PoC work has so far been entirely on the confidential-inference side and has not yet touched call handling.

9 Next Steps

Table 4 sets out the remaining milestones toward the end-to-end live PoC (M7). Each milestone in Table 2 and Table 4 together is independently demonstrable and citable in the thesis write-up even if time runs out before M7, which is the reason the work has been staged this way.

Stage	Planned work
M4	Recompile the verified ONNX/Relax model using TVM's C code-generation target (rather than the LLVM JIT target used for verification in M3) to produce standalone, dependency-free C source suitable for the OP-TEE TA build environment.
M5	Create a new trusted application, structured after the verified `hello_world` example (M2), exposing a single command (`TA_CMD_RUN_INFERENCE`) that runs one forward pass of the compiled model on a fixed-length input buffer and returns a spoof/bona fide score.
M6	Write a client application that sends one static, known test vector to the TA and confirms the returned score matches the PyTorch/ONNX/TVM reference score obtained in M3, closing the correctness loop from trained model through to in-TEE inference.
M7	Integrate the client application with the Asterisk rolling audio buffer so that live call audio is windowed, fed to the trusted application, and a spoof/bona fide decision is produced during an active call – the end-to-end live PoC.

Table 4: Remaining milestones toward the end-to-end live PoC.

10 Summary

The confidential-inference phase of the thesis has, to date, established that the intended architecture is feasible on the standard OP-TEE-supported emulation platform: the secure/normal-world split boots and passes the full official test suite (M1), the exact CA↔TA communication pattern the inference pipeline will use has been validated with a working example (M2), and the trained RawTFNet-16 model has been exported and compiled through the only viable path for running inside a trusted application, with its output verified numerically identical to the original PyTorch model to four decimal places (M3). The remaining work (M4–M7) is scoped as linking this already-verified model into an already-validated TA/CA pattern, and then, separately, wiring that pipeline to live Asterisk audio – rather than as open research risk. The two open items in Section 8, and the QEMU-vs-Pi4 timing caveat in Section 6, are the main points flagged for discussion at this stage.